

Lezione 3: Esercitazione Pratica

Streams

Michele Gentile

12 Maggio 2026

0 Ripasso di Teoria

A. Cos'è uno Stream e la Sorgente

🔗 Concetto in breve

Uno Stream non è una struttura dati (come un Array o una Lista). È un flusso che trasporta elementi da una sorgente (collezione, array...) attraverso una pipeline di funzioni.

💡 Tip: Analogia col mondo reale

Un nastro trasportatore in una fabbrica di cioccolato. Il nastro non possiede i cioccolatini (quelli stanno nel magazzino/Lista), ma li sposta affinché possano essere lavorati uno alla volta.

In Java, uno Stream segue tre fasi: **Sorgente** → **Operazioni Intermedie** → **Operazione Terminale**.

- **Sorgente:** Da dove partono i dati (es. `lista.stream()`).
- **Non-distruttivo:** Lo Stream non modifica mai la sorgente originale. Crea un nuovo risultato.
- **Lazy (Pigro):** Lo Stream non inizia a muovere i dati finché non vede un'operazione terminale alla fine della linea.

```
List<String> nomi = Arrays.asList("Marco", "Anna", "Luca");  
// Creazione dello Stream dalla sorgente (Lista)  
Stream<String> flussoNomi = nomi.stream();
```

B. Operazioni Intermedie: La Pipeline

🔗 Concetto in breve

Sono istruzioni che dicono allo Stream come trasformare o filtrare i dati che passano sul nastro. Ogni operazione intermedia restituisce un nuovo Stream.

- `filter(Predicate)`: Accetta una condizione (lambda) e lascia passare solo gli elementi che la soddisfano.
- `map(Function)`: Trasforma ogni elemento in qualcos'altro (es. Stringa → n di caratteri).
- `distinct()`: Elimina i duplicati.

💡 Tip: Analogia Estesa

Immaginiamo che lungo il nastro trasportatore ci siano degli operai:

- **Filter (Il Selezionatore)**: Scarta i cioccolatini che sono rotti o non conformi.
- **Map (Il Decoratore)**: Prende un cioccolatino bianco e lo ricopre di cioccolato fondente (trasformazione).
- **Sorted (L'Ordinatore)**: Mette in fila i cioccolatini dal più piccolo al più grande.

```
List<String> nomi = Arrays.asList("Marco", "Anna", "Luca", "Andrea")

nomi.stream()
    .filter(n -> n.startsWith("A")) // Passano solo Anna e Andrea
    .map(String::toUpperCase)       // Diventano ANNA e ANDREA
    .sorted()                       // Vengono ordinati
    // ... manca l'operazione terminale per vedere i frutti!
```

C. Operazioni Terminali: Il Risultato Finale

🔗 Concetto in breve

È il punto finale della pipeline. Una volta eseguita, lo Stream viene consumato e non può più essere riutilizzato.

💡 Tip: Analogia col mondo reale

È la stazione di imballaggio. Qui i cioccolatini vengono messi in una scatola o vengono contati per essere spediti. Una volta messi nella scatola, il nastro trasportatore è vuoto.

Senza un'operazione terminale, le operazioni intermedie non verranno mai eseguite.

- `collect(Collectors.toList())`: Riunisce i risultati in una nuova lista.
- `forEach(Consumer)`: Esegue un'azione per ogni elemento (es. stampa).
- `count()`: Restituisce il numero di elementi che hanno superato i filtri.
- `reduce()`: Combina tutti gli elementi in un unico valore (es. somma di numeri).

```
List<Integer> numeri = Arrays.asList(1, 2, 3, 4, 5);
// Somma dei quadrati dei numeri pari
int somma = numeri.stream()
    .filter(n -> n % 2 == 0) // Filtra: 2, 4
```

```
.map(n -> n * n)           // Trasforma: 4, 16
.reduce(0, Integer::sum);    // Somma: 20

System.out.println(somma); // Output: 20
```

Esercizi di verifica

⚠ Attenzione: Trappole Comuni

- **Riutilizzo dello Stream:** Molti provano a usare lo stesso oggetto Stream due volte. Una volta che si chiama un'operazione terminale, lo Stream è "chiuso". Se serve di nuovo, va ricreato dalla sorgente.
- **Dimenticare l'Operazione Terminale:** Scrivere una pipeline lunghissima e poi meravigliarsi che non succeda nulla. Senza il "tappo" finale, Java non muove un dito.

🔗 Domande di Verifica

1. *"Se ho una lista di 1000 nomi e applico un filter e un map, la lista originale viene modificata?"*
Risposta: No, gli Stream sono immutabili rispetto alla sorgente.
2. *"Cosa succede se scrivo `stream().filter(n -> n > 10)` senza aggiungere altro?"*
Risposta: Non succede nulla, il filtro non viene nemmeno eseguito perché manca l'operazione terminale.
3. *"Qual è la differenza tra map e filter in termini di numero di elementi in uscita?"*
Risposta: `filter` può ridurre il numero di elementi, `map` mantiene lo stesso numero ma può cambiarne il tipo/forma.

💡 Tip: Punti Chiave da Ricordare

- Lo Stream è un flusso, non una collezione.
- **Lazy Evaluation:** Le operazioni intermedie partono solo quando arriva quella terminale.
- **Pipeline:** Si legge come una frase (Sorgente → Filtra → Trasforma → Raccogli).
- **Una sola vita:** Uno Stream si usa una volta sola.

1 Analisi Funzionale di una Collezione di Animali

💡 Tip: Ricordiamo l'esercizio Animali

1. Progettare una gerarchia di classi dei seguenti animali, ciascuno con determinate caratteristiche:

- Uccello (vola, becca)
 - Pinguino (becca, nuota)
 - Aquila (vola, becca)
- Pesce (nuota)
 - Pesce volante (nuota, vola)
- Cane (salta, corre, fedele a, domestico)
- Felino (salta, corre, fa le fusa)
 - Gatto (salta, corre, fa le fusa, domestico)
- Uomo (salta, corre, pensa, nuota, vola?)

1. Si riprenda la gerarchia della classe **Animale**. Si crei una lista di oggetti di tipo *Animale* (es. Cani, Gatti, Pesci).
2. Utilizzando gli **Stream**, si implementi un metodo che restituisca una lista contenente solo i nomi degli animali che hanno la caratteristica di essere **domestici**.
3. Si scriva una pipeline che filtri gli animali in grado di **volare** e ne stampi il nome in maiuscolo, ordinandoli alfabeticamente.
4. Utilizzando lo stream, calcolare il numero totale di animali che possiedono la capacità di **nuotare**.

🔗 Obiettivo: Filter, Map e Count

L'esercizio mira a familiarizzare con le operazioni intermedie di filtraggio e trasformazione, tipiche della programmazione dichiarativa.

💡 Tip: Predicati e Metodi

Per verificare le caratteristiche (es. "vola"), potete usare riferimenti a metodi (*method references*) come `Animale::puoVolare` se il metodo restituisce un booleano.

2 Statistiche Avanzate sulla Raccolta di Canzoni

💡 Tip: Ricordiamo l'esercizio Canzoni

1. Si definisca una classe **Canzone** i cui oggetti sono costruiti con nome e autore
2. Si crei una classe **Raccolta** che contiene un insieme di canzoni
 - In una prima versione, si utilizzi `HashSet` per memorizzare l'insieme di canzoni
 - Si implementi correttamente `equals` e `hashCode` in modo da evitare duplicati di canzoni sostanzialmente uguali (ovvero con lo stesso nome e autore)
 - La classe è dotata di un metodo `toString` che restituisce la rappresentazione sotto forma di stringa dell'insieme di canzoni (analogamente, le canzoni sono dotate dello stesso metodo)
 - Si implementi quindi la classe **RaccoltaOrdinata** che utilizza `TreeSet` per mantenere un ordine sulle canzoni
 - Implementare un ordine sulle canzoni basato sul nome della canzone e, se questo è uguale, sul nome dell'autore
3. Si estenda la classe **Canzone** aggiungendo un campo intero `durataSecondi`.
4. Data una **Raccolta** di canzoni, si utilizzi uno stream per calcolare la **durata totale** di tutte le canzoni presenti utilizzando `mapToInt` e `sum`.
5. Implementare un metodo che restituisca la canzone con la durata massima utilizzando l'operatore `max` e un `Comparator`.
6. Si crei una stringa che contenga tutti i titoli delle canzoni della raccolta, separati da una virgola, utilizzando `Collectors.joining(", ")`.

🔗 Obiettivo: Stream Numerici e Riduzione

L'uso di `IntStream` e dei raccoglitori (`Collectors`) permette di aggregare dati in modo efficiente senza cicli `for` espliciti.

⚠️ Attenzione: Gestione degli Optional

Il metodo `max()` restituisce un `Optional`. Ricordate di gestire il caso in cui la raccolta sia vuota usando `orElse` o `ifPresent`.

3 Raggruppamento e Partizionamento Dati

1. Si definisca una classe **Prodotto** con campi `nome`, `categoria` (`String`) e `prezzo` (`double`).
2. Data una lista di prodotti, utilizzare `Collectors.groupingBy` per ottenere una `Map` dove la chiave è la categoria e il valore è la lista dei prodotti appartenenti a quella categoria.
3. Utilizzare `Collectors.partitioningBy` per dividere i prodotti in due gruppi: quelli con prezzo superiore a 50€ e quelli con prezzo inferiore o uguale.

4. Per ogni categoria, calcolare il prezzo medio dei prodotti utilizzando `Collectors.averagingDouble`.

🔗 Obiettivo: Collectors e Map

Capire come trasformare un flusso lineare di dati in strutture dati gerarchiche o aggregate come le `Map`.

💡 Tip: Downstream Collectors

Il metodo `groupingBy` può accettare un secondo parametro (un altro collector) per eseguire operazioni sui valori raggruppati, come contare gli elementi o sommare i prezzi.

4 Pipeline Complesse: Gestione Ordini

1. Si crei una classe **Ordine** che contiene una lista di **Prodotti** e una data di acquisto.
2. Data una lista di ordini, si utilizzi `flatMap` per ottenere uno stream di tutti i singoli prodotti acquistati in tutti gli ordini.
3. Dalla lista globale di prodotti ottenuta, filtrare i duplicati (prodotti uguali) e restituire una lista di nomi di prodotti unici ordinati per prezzo decrescente.
4. Trovare il prodotto più costoso mai acquistato in un ordine effettuato nell'anno 2025.

🔗 Obiettivo: FlatMap e Ordinamento Personalizzato

Saper "appiattare" strutture annidate (liste dentro liste) e manipolare i dati risultanti attraverso pipeline multi-livello.

⚠️ Attenzione: Immutabilità

Ricordate che gli `Stream` non modificano la sorgente originale. Le operazioni di `sorted()` o `filter()` creano nuove viste dei dati; per salvare il risultato è sempre necessario un'operazione terminale come `toList()`.